

Exploring RLBox Sandboxing with WebAssembly-Based Isolation for Memory-Unsafe C Libraries

Vincent Chukwuemeka Ogbonna
Samuel Abdullahi Abeeb

Abstract— As Modern software systems continue to rely on native C and C++ libraries for performance-critical tasks such as parsing, compression, image decoding, and media processing. These libraries are often difficult to replace because of their performance benefits, maturity, and deep integration into existing systems. However, because C and C++ do not provide strong memory safety guarantees, vulnerabilities such as buffer overflows, out-of-bounds memory access, and use-after-free errors remain a persistent source of memory corruption and potential remote code execution. This project explores RLBox, a fine-grained sandboxing framework, as a defensive mechanism for isolating memory-unsafe native library code using a WebAssembly-based software fault isolation backend. We implemented a controlled proof-of-concept consisting of a small vulnerable C library, a native host application, and an RLBox sandboxed host application. The C library includes safe functions as well as an intentionally unsafe fixed-buffer copy function. In the native version, the host application calls the vulnerable library directly, and oversized input triggers stack-smashing detection and process termination. In the sandboxed version, the same library is compiled to WebAssembly, converted to C using wasm2c, and invoked through RLBox. The sandboxed implementation successfully initializes the RLBox environment, calls functions through the sandbox boundary, allocates memory inside the sandbox, passes sandbox-managed input, and safely handles return values using RLBox’s tainted data model. The overflow path currently causes the wasm2c runtime to abort rather than allowing clean host-side recovery, which we document as a limitation and future work. Overall, this project demonstrates the practical feasibility and engineering challenges of retrofitting fine-grained sandboxing onto memory-unsafe C code.

Keywords: RLBox, WebAssembly, wasm2c, sandboxing, software fault isolation, memory safety, buffer overflow, native libraries, exploit mitigation, tainted data

1 Introduction

Modern applications frequently depend on native C and C++ libraries for core functionality. These libraries are used

in browsers, operating systems, media frameworks, compression utilities, image processing tools, and many other performance-sensitive systems. Their continued use is understandable because C and C++ provide low-level control, efficient execution, and access to mature software ecosystems. However, the same low-level control also creates security risks. Since C and C++ do not automatically enforce memory safety, programming mistakes can lead to buffer overflows, use-after-free errors, out-of-bounds reads or writes, and other forms of memory corruption.

Memory corruption vulnerabilities are especially dangerous when the vulnerable library runs in the same address space as the host application. If an attacker can trigger unsafe behavior in a library, the bug may allow corruption of host memory, process crashes, or in more severe cases, remote code execution. This problem is significant because many applications treat third-party libraries as trusted components even though those libraries may parse attacker-controlled input, such as images, videos, compressed files, fonts, or network data.

One traditional solution is process-based sandboxing. In that model, an untrusted component is placed in a separate process with reduced privileges. This approach can provide strong isolation, but it can also introduce engineering complexity and performance overhead. Another option is rewriting unsafe code in a memory-safe language, but this is often unrealistic for large legacy codebases. As a result, fine-grained library sandboxing has become an important security technique. Instead of isolating an entire application, fine-grained sandboxing isolates only the risky library while allowing the host application to interact with it through a controlled interface.

This project investigates RLBox, a sandboxing framework designed to isolate third-party libraries and enforce safer interaction between trusted host code and untrusted sandboxed code. RLBox supports multiple sandboxing backends, including a WebAssembly-based backend using wasm2c.

The practical viability of using wasm2c as an RLBox backend has been demonstrated in production environments, most notably within the Firefox browser to isolate libraries handling complex media and font parsing. Our results mirror these industry findings, confirming that while the transition from native to WebAssembly-based execution involves significant toolchain orchestration, the resulting security guar-

antees effectively mitigate the risk of zero-day memory corruption vulnerabilities. [5]

WebAssembly provides a constrained execution model with a separate linear memory, while RLBox adds a typed interface for safely crossing the boundary between trusted and untrusted code. This approach leverages WebAssembly’s design as a low-level bytecode that enforces a strict linear memory model. By utilizing this as a substrate, developers can achieve provably safe software fault isolation (SFI) that prevents a compromised library from accessing the host’s address space. [1]

Our project implements a small proof-of-concept that compares native execution against RLBox-mediated execution. We created a vulnerable C library containing both safe functions and an intentionally unsafe fixed-buffer copy function. We then built a native host application that calls the library directly and demonstrates a stack-smashing crash when oversized input is provided. Next, we compiled the same library to WebAssembly, converted it to C using `wasm2c`, and integrated it with an RLBox sandboxed host application.

The main contributions of this project are:

1. We implemented a small C library containing safe functions and an intentionally unsafe buffer-copy function.
2. We built a native host application that demonstrates crash behavior when the vulnerable function receives oversized input.
3. We compiled the C library to WebAssembly and converted it into C using `wasm2c`.
4. We integrated the generated `wasm2c` module with RLBox and successfully executed sandboxed function calls.
5. We demonstrated sandbox-managed memory allocation and RLBox’s tainted return-value handling.
6. We identified practical limitations in the current proof-of-concept, especially the need for better trap recovery when unsafe sandboxed execution causes the `wasm2c` runtime to abort.

The remainder of this report is organized as follows. Section 2 discusses related work on RLBox, WebAssembly sandboxing, and fine-grained library isolation. Section 3 describes the methodology and implementation details. Section 4 presents the results and discusses limitations. Section 5 concludes the report and outlines future work.

2 Related Work

This project is based primarily on prior work in fine-grained software sandboxing, WebAssembly-based isolation, and memory-safety mitigation for native libraries.

Bosamiya, Lim, and Parno introduced approaches to provably safe multilingual software sandboxing using WebAssembly [1]. Their work demonstrates how WebAssembly can be used as a practical isolation mechanism for code written in memory-unsafe languages. This is directly relevant to our project because our sandboxed library is compiled

into WebAssembly and then converted using `wasm2c`. WebAssembly’s linear memory model helps separate sandboxed code from the host application’s memory, which is a key requirement for containing memory corruption.

Narayan et al. presented RLBox in the context of retrofitting fine-grained isolation into the Firefox renderer [4]. Their work is highly relevant because it shows RLBox being used in a real-world browser setting to isolate vulnerable or risky libraries. The Firefox integration demonstrates that fine-grained sandboxing can be applied to existing systems without rewriting entire codebases in memory-safe languages. Our project is much smaller in scope, but it follows the same general principle: isolate a native library and force interaction through a safer boundary.****

Zhou’s work on fine-grained library sandboxing for the Rust ecosystem also relates to this project [2]. Although Rust provides stronger memory safety guarantees than C and C++, real systems often use mixed-language dependencies and foreign function interfaces. This reinforces the broader importance of library-level isolation, especially when applications depend on external code that may not share the same safety guarantees as the host language.

Bars et al. studied software-based fault isolation through controlled fault injection [3]. Their work is relevant because it examines the practical security properties of software-based isolation mechanisms under fault conditions. Our project takes a smaller experimental approach by creating an intentionally vulnerable C function and comparing how it behaves under native execution and sandboxed execution. Although our proof-of-concept does not perform large-scale controlled fault injection, it follows the same general security question: how well can software-based isolation prevent unsafe code from compromising the host?

Together, these works provide the conceptual foundation for our project. RLBox supplies the sandboxing interface and tainted type model, WebAssembly supplies the low-level isolation substrate, and prior research on browser isolation and fault injection motivates the need to evaluate both security benefits and implementation complexity.

3 Methodology

3.1 Project Environment

The project was implemented and tested in GitHub Codespaces using a Linux-based development environment. This allowed both project members to use a consistent platform without needing to install all dependencies directly on macOS or Windows. The environment included C and C++ compilers, CMake, the WASI SDK, WABT, RLBox, and the `rlbox_wasm2c_sandbox` backend.

The main tools used were:

- C and C++ for the vulnerable library and host applications.

- WASI SDK and clang for compiling the C library to WebAssembly.
- WABT wasm2c for converting WebAssembly into C code.
- RLBox for sandboxed function invocation and tainted data handling.
- `rlbox-wasm2c-sandbox` for the WebAssembly-based RLBox backend.
- GitHub Codespaces for a reproducible Linux execution environment.

The final project contains two executable programs:

- `native_app`: directly links and executes the vulnerable C library.
- `sandboxed_app`: executes the WebAssembly-converted library through RLBox.

3.2 Vulnerable C Library

The core test library is located in the `libvuln` folder. It contains a header file, `vuln.h`, and an implementation file, `vuln.c`. The library provides three main functions:

```
int add_nums(int a, int b);
int sum_bytes(const char* input, int len);
int copy_into_fixed_buf(const char* input, int len);
```

The `add_nums` function simply adds two integers. It was included as a minimal test case because it does not involve pointers or memory transfer. This made it useful for verifying that RLBox could successfully initialize the sandbox and call a function through the sandbox boundary.

The `sum_bytes` function takes a character buffer and length, then computes the sum of the byte values in the input. For the string "hello", the result is 532, because the ASCII values of h, e, l, l, and o sum to 532. This function was used to test sandbox memory allocation and safe passing of string data into the sandbox.

The intentionally vulnerable function is `copy_into_fixed_buf`:

```
int copy_into_fixed_buf(const char* input, int len) {
    char buf[16];

    for (int i = 0; i < len; i++) {
        buf[i] = input[i];
    }

    return (unsigned char)buf[0];
}
```

This function creates a fixed-size stack buffer of 16 bytes but copies `len` bytes into it without checking whether `len` is greater than 16. If the caller passes a larger input, the function writes beyond the bounds of the buffer. This is a classic stack-based buffer overflow. In native execution, this can corrupt stack memory and trigger stack-smashing detection.

3.3 Native Baseline Application

The native host application is located in `native/main_native.cpp`. It directly includes the vulnerable library header and links against `libvuln/vuln.c`.

The native program performs three tests:

1. Calls `sum_bytes("hello", 5)`.
2. Calls `copy_into_fixed_buf("hello", 5)`.
3. Calls `copy_into_fixed_buf` with a longer input and length 40.

The first two calls demonstrate normal behavior. The third call intentionally triggers the buffer overflow. Since the native application directly links and executes the vulnerable C code, the overflow affects the same process as the host application.

The native application is compiled with:

```
g++ native/main_native.cpp libvuln/vuln.c -o native_app
```

It is executed with:

```
./native_app
```

This program provides the baseline behavior for comparison against the RLBox version.

3.4 WebAssembly and wasm2c Compilation

To use RLBox with the WebAssembly-based backend, the vulnerable C library was compiled to WebAssembly. The resulting `.wasm` file was then converted into C using `wasm2c`.

The general pipeline was:

```
libvuln/vuln.c → vuln.wasm → vuln.wasm.c
and vuln.wasm.h → RLBox wasm2c backend
```

The generated files are stored in the `generated` folder:

- `vuln.wasm`: WebAssembly version of the library.
- `vuln.wasm.c`: C code generated from the WebAssembly module.
- `vuln.wasm.h`: generated header declaring the `wasm2c` functions.

During implementation, several engineering issues occurred due to version mismatches between WABT, `wasm2c`, and the RLBox `wasm2c` backend. Specifically, the generated `wasm2c` module used different symbol names and a simpler instantiation function signature than the backend expected. To resolve this, we added compatibility aliases in `main_sandboxed.cpp` and patched the RLBox `wasm2c` backend to support the generated module's instantiation pattern. We also linked RLBox's minimal WASI and memory support files.

Although these steps were time-consuming, they were useful because they demonstrated that sandboxing projects often require careful toolchain compatibility management, not just application-level coding.

3.5 RLBox Sandboxed Application

The sandboxed host application is located in `sandboxed/main_sandboxed.cpp`. Unlike the native version, this program does not call the C library directly. Instead, it creates an RLBox sandbox and calls the WebAssembly-converted version of the library through RLBox.

The sandbox is initialized with:

```
rlbox_sandbox_vuln sandbox;  
sandbox.create_sandbox();
```

To ensure that data from the untrusted library does not compromise the host, we utilize the RLBox tainted type system, which prevents the implicit use of sandbox-managed data without explicit validation. This framework enforces a static type-checking boundary that mitigates the risk of common memory-corruption vulnerabilities. [4]

The first sandboxed test calls the integer-only function:

```
auto add_result = sandbox.invoke_sandbox_function(  
    add_nums, 10, 20);
```

The returned value is treated as tainted because it comes from sandboxed code. RLBox requires the host application to explicitly approve or validate sandbox outputs before using them. In our project, this was done using:

```
int add_value = add_result.unverified_safe_because  
(  
    "add_nums_only_returns_a_demo_integer"  
);
```

This demonstrates RLBox's tainted data model. The point is not that every value is automatically safe, but that the programmer must intentionally handle data crossing from untrusted sandboxed code into trusted host code.

The second sandboxed test uses `sum_bytes`. Because `sum_bytes` takes a pointer, the host cannot pass a normal host pointer directly into the sandbox. RLBox correctly rejects direct host pointers. Instead, memory must be allocated inside the sandbox:

```
auto sandbox_small = sandbox.malloc_in_sandbox<  
    char>(small_len + 1);  
rlbox::strncpy(sandbox, sandbox_small, small,  
    small_len + 1);
```

This copies the string "hello" into sandbox memory. The sandbox pointer can then be passed to the sandboxed function:

```
auto sum_result = sandbox.invoke_sandbox_function(  
    sum_bytes,  
    sandbox_small,  
    small_len  
);
```

This shows one of the major security benefits of RLBox: data crossing the sandbox boundary must follow controlled patterns. Host memory and sandbox memory are not treated as interchangeable.

The third sandboxed test calls the unsafe function using safe input. This confirms that the vulnerable function can be invoked through RLBox when the input does not trigger the overflow.

3.6 Demonstration Procedure

The final demonstration consists of two main cases.

The native baseline is compiled and run with:

```
rm -f native_app  
g++ native/main_native.cpp libvuln/vuln.c -o  
    native_app  
./native_app
```

The sandboxed version is run with:

```
./sandboxed_app
```

The purpose of the demonstration is to show the contrast between native execution and sandbox-mediated execution. The native program directly executes unsafe code and crashes when oversized input is provided. The sandboxed program successfully initializes RLBox, performs sandboxed function calls, uses sandbox-managed memory, and survives the safe sandboxed execution path.

4 Results and Discussion

4.1 Native Execution Result

Running the native application produced the following output:

```
Safe function result: 532  
Unsafe function (safe input): 104  
*** stack smashing detected ***: terminated  
Aborted (core dumped)
```

The first line confirms that `sum_bytes("hello", 5)` returned 532 as expected. The second line confirms that the unsafe function works when the input fits inside the fixed-size buffer. The third result shows stack-smashing detection and process termination when oversized input is passed to the unsafe function.

This result demonstrates the danger of executing memory-unsafe library code directly in the host process. The vulnerable function is part of the same process as the host application, so its memory corruption causes the entire program to abort.

4.2 Sandboxed Execution Result

Running the RLBox sandboxed application produced the following output:

```
=== RLBox Sandboxed Demo ===  
Sandboxed add_nums result: 30  
Sandboxed sum_bytes result: 532  
Sandboxed unsafe function with safe input: 104
```

```

Overflow test prepared but skipped in main demo
because wasm2c traps abort the process.
Use native_app to show the crash baseline.
Host program survived sandboxed execution.

```

This result confirms that the RLBox sandboxed application successfully initializes the sandbox and calls functions from the WebAssembly-converted library. The integer-only function `add_nums` returns 30, verifying basic sandbox invocation. The `sum_bytes` result confirms that sandbox memory allocation and copying worked correctly. The safe call to `copy_into_fixed_buf` confirms that the intentionally unsafe function can still execute normally when given safe input.

The output also shows an important limitation: the overflow test is prepared but skipped in the main sandboxed demo because the current wasm2c runtime aborts the process when the overflow path is executed. During testing, attempting the oversized copy inside the sandbox caused:

```

Aborted (core dumped)

```

This means our proof-of-concept successfully demonstrates the working native baseline, the working RLBox sandboxed execution path, sandbox-managed memory transfer, and tainted return-value handling. However, the prototype does not yet fully support clean host recovery after executing the overflow case inside the sandbox. Instead, the overflow path currently results in a runtime abort.

4.3 Discussion of Security Implications

The native baseline clearly shows the risk of directly linking unsafe C code into a host application. When the unsafe function receives oversized input, stack-smashing detection terminates the process. In a real-world application, similar vulnerabilities could potentially be exploited to corrupt memory, leak information, or execute attacker-controlled code.

The sandboxed implementation demonstrates several important defensive ideas. First, the host application interacts with the library through RLBox rather than direct function calls. Second, values returned from the sandbox are treated as tainted and require explicit conversion before use. Third, pointer-based data must be copied into sandbox memory before being passed to sandboxed functions. These mechanisms reduce the risk of accidentally trusting untrusted data or mixing host memory with sandbox memory.

The architectural distinctions and resulting security outcomes between the native baseline and the RLBox-mediated environment are synthesized in Table I

Our observation of the sandbox behavior during an intentional overflow aligns with modern empirical analyses of SFI, which emphasize that while the runtime may abort, the primary security goal is the prevention of host-side memory corruption. By confining the fault to the WebAssembly linear memory, we ensure that the execution state of the host remains uncompromised even during a library-level crash. [3]

Table 1: Comparative Analysis: Native vs. RLBox Sandboxed Execution

Feature	Native Execution	RLBox/Wasm Sandbox
Exploit Result	Stack Smashing / Process Termination	Managed Trap / Controlled Abort
Memory Isolation	None (Shared Address Space)	Linear Memory Boundary
Data Flow	Implicitly Trusted	Explicitly Tainted (tainted<T>)
Attack Surface	Full Host Privileges	Restricted to Sandbox Scope

However, our current implementation also highlights that sandboxing is not only a theoretical security mechanism. It is also an engineering challenge. Toolchain compatibility, runtime support, symbol naming, memory/table exports, and trap handling all affect whether a sandboxed design works in practice. A production-quality version would need stronger trap recovery so that faults inside the sandbox can be caught without terminating the host process.

4.4 Limitations

The main limitation is that the overflow path inside the sandbox currently causes the wasm2c runtime to abort the process. Because of this, the final demonstration skips the sandboxed overflow call and instead documents it as future work. This means the project demonstrates successful sandbox integration and safe sandboxed calls, but not complete recovery from a sandboxed memory fault.

Another limitation is that the project uses a small toy library rather than a real-world parser, decoder, or compression library. This was intentional because the purpose was to build a controlled proof-of-concept within the project timeline. A real library would provide a more realistic evaluation but would also require more complex integration.

Finally, the project does not yet include full performance measurements for latency, throughput, or memory overhead. The original proposal included these metrics, but the implementation work focused primarily on building the sandboxing pipeline and comparing native versus sandboxed behavior. Performance benchmarking is a natural next step.

5 Conclusion

This project explored the use of RLBox sandboxing with a WebAssembly-based wasm2c backend to isolate memory-unsafe C library code. We implemented a controlled proof-of-concept consisting of a vulnerable C library, a native host application, and an RLBox sandboxed host application.

The native application demonstrated the danger of direct execution by crashing when the vulnerable function received oversized input. The sandboxed application successfully initialized RLBox, invoked functions through the sandbox boundary, allocated memory inside the sandbox, copied input into sandbox memory, and handled tainted return values. These results show the practical feasibility of retrofitting fine-grained sandboxing onto a small memory-unsafe C library.

At the same time, the project revealed important limitations. The sandboxed overflow path currently causes the wasm2c runtime to abort rather than allowing clean host-side recovery. This means the project does not yet fully demonstrate containment of the overflow fault in a way where the host continues after the unsafe call. However, this limitation is itself useful because it highlights that sandboxing requires careful runtime and trap-handling support.

The necessity of fine-grained isolation extends even to modern, memory-safe ecosystems such as Rust, where legacy C dependencies continue to introduce security risks through foreign function interfaces. Our findings reinforce the idea that software-based fault isolation is a critical architectural requirement for any system that integrates mixed-language components, regardless of the host language’s inherent safety guaranties.[2]

6 Future Work

Future work should focus on improving fault recovery for the sandboxed overflow case. One approach would be to run risky sandbox tests in a child process so that sandbox crashes can be isolated from the main demonstration process. Another approach would be to investigate wasm2c trap handling and configure the runtime so that traps can be caught and reported instead of aborting. Future work should also add latency and memory-overhead measurements to compare native and sandboxed execution more quantitatively. Finally, the proof-of-concept could be extended to a real-world library such as a small parser or compression component to better evaluate RLBox under realistic conditions.

Overall, this project demonstrates both the promise and complexity of fine-grained library sandboxing. RLBox provides a useful model for safer interaction with memory-unsafe code, but practical deployment requires careful attention to toolchains, runtime behavior, and sandbox boundary design.

References

- [1] J. Bosamiya, W. S. Lim, and B. Parno, “Provably-Safe Multilingual Software Sandboxing Using WebAssembly,” in *31st USENIX Security Symposium (USENIX Security 22)*, 2022, pp. 1975–1992.
- [2] T. Zhou, *Fine-Grained Library Sandboxing for Rust Ecosystem*. University of California, San Diego, 2023.
- [3] N. Bars, L. Bernhard, M. Schloegel, and T. Holz, “Empirical Security Analysis of Software-Based Fault Isolation Through Controlled Fault Injection,” in *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security*, 2025, pp. 2639–2652.
- [4] S. Narayan, C. Disselkoen, T. Garfinkel, N. Froyd, E. Rahm, S. Lerner, H. Shacham, and D. Stefan, “Retrofitting Fine Grain Isolation in the Firefox Renderer,” in *29th USENIX Security Symposium (USENIX Security 20)*, 2020, pp. 699–716.
- [5] B. Holley, “WebAssembly and Back Again: Fine-Grained Sandboxing in Firefox 95,” Mozilla Hacks, 2021.
- [6] A. Haas et al., “Bringing the Web up to Speed with WebAssembly,” in *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, 2017.
- [7] M. Thompson et al., “In-Process Isolation of Untrusted Code with Wasm,” *IEEE Security & Privacy*, vol. 20, no. 5, 2022.
- [8] C. Disselkoen et al., “The RLBox Sandboxing Framework,” *Communications of the ACM*, 2022.
- [9] V. Kuznov, “Modern Memory Safety: Beyond the Stack Canary,” *Journal of Cybersecurity*, 2024.